

IME 775 — Lecture 18

Optimization Algorithms and Regularization

1. Review: The Optimization Update Rule

In each iteration t , we update parameters using:

$$\vec{w}_{t+1} = \vec{w}_t - \Delta \vec{w}_t, \quad \vec{b}_{t+1} = \vec{b}_t - \Delta \vec{b}_t$$

The simplest update (vanilla SGD) computes:

$$\Delta \vec{w}_t = \eta \nabla_{\vec{w}} L(\vec{w}_t, \vec{b}_t)$$

where η is the learning rate. This chapter introduces more sophisticated strategies for computing $\Delta \vec{w}_t$ that lead to faster and more reliable convergence.

2. Momentum

The Problem with Vanilla SGD

In high dimensions, loss surfaces resemble rough canyons. Minibatch gradient estimates are **noisy** — they don't always point directly toward the minimum. The gradient at any point has:

- A **useful downhill component** (roughly shared across iterations)
- A **noisy component** (random direction, varies per iteration)

If we average over time, the useful components reinforce while the noise cancels.

Momentum Update

$$\Delta \vec{w}_t = \beta \Delta \vec{w}_{t-1} + \eta \nabla_{\vec{w}} L(\vec{w}_t, \vec{b}_t)$$

where $\beta \in (0, 1)$ is the momentum coefficient (typically 0.9).

Mental model: A ball rolling downhill accumulates velocity. On flat spots, momentum carries it through. On noisy terrain, the consistent downhill direction gets amplified while cross-hill oscillations get damped.

Unrolling the Recursion

$$\Delta \vec{w}_t = \eta \nabla L_t + \eta \beta \nabla L_{t-1} + \eta \beta^2 \nabla L_{t-2} + \dots + \eta \beta^t \nabla L_0$$

The sum of weights is $\eta(1 + \beta + \beta^2 + \dots) = \frac{\eta}{1-\beta}$. Since this doesn't sum to 1, momentum gives a weighted sum — not a weighted average. This is corrected in Adam.

Workout: With $\eta = 0.01$ and $\beta = 0.9$, what is the effective amplification factor compared to vanilla SGD?

Solution: The sum of weights is $\frac{\eta}{1-\beta} = \frac{0.01}{0.1} = 0.1$. Without momentum, the effective step is $\eta = 0.01$. With momentum, when gradients are consistently aligned, the effective step approaches 0.1, a **10× amplification**.

3. Nesterov Accelerated Gradients

The Problem with Momentum

Momentum can **overshoot** the minimum. Near the bottom of the loss surface, the accumulated velocity may carry the optimization past the minimum and up the other side.

Nesterov's Fix: Look Ahead

Instead of computing the gradient at the current position, compute it at the **estimated destination** (where momentum would take us):

$$\Delta \vec{w}_t = \beta \Delta \vec{w}_{t-1} + \eta \nabla_{\vec{w}} L(\vec{w}_t - \beta \Delta \vec{w}_{t-1}, \vec{b}_t - \beta \Delta \vec{b}_{t-1})$$

Why it works:

- **Far from minimum:** The gradient at the estimated destination is similar to the gradient at the current point → behaves like standard momentum
- **Near minimum (about to overshoot):** The estimated destination is on the opposite side of the minimum → the gradient there **opposes** the momentum direction → the weighted average is smaller → overshooting is reduced

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01,  
momentum=0.9, nesterov=True)
```

4. AdaGrad: Per-Parameter Learning Rates

The Problem with Global Learning Rate

A single learning rate η treats all dimensions equally, but the loss surface is not symmetric. Some dimensions have steep gradients (need small steps), while others have shallow gradients (need large steps).

AdaGrad State and Update

AdaGrad maintains a per-parameter accumulator of squared gradients:

$$\vec{s}_t = |\nabla L_t|^2 + \vec{s}_{t-1}$$

The update divides the gradient by the square root of this accumulator:

$$\Delta \vec{w}_t = \frac{\eta}{\sqrt{\vec{s}_t} + \epsilon} \odot \nabla L_t$$

where ϵ is a small constant preventing division by zero and $\odot$ is element-wise multiplication.

Effect: Dimensions with historically large gradients get smaller learning rates, and vice versa. This adapts the step size **per parameter**.

AdaGrad's Fatal Flaw

The accumulator $\vec{s}_t$ only grows — it never decreases. Over many iterations, the effective learning rate shrinks toward zero for **all** parameters, and training stalls.

5. RMSProp: Fixing AdaGrad

RMSProp uses an **exponential moving average** of squared gradients instead of the cumulative sum:

$$\vec{s}_t = (1 - \beta) |\nabla L_t|^2 + \beta \vec{s}_{t-1}$$

Key insight: The sum of the weights approaches 1 as $t \rightarrow \infty$:

$$(1 - \beta)(1 + \beta + \beta^2 + \dots) = (1 - \beta) \cdot \frac{1}{1 - \beta} = 1$$

This means $\vec{s}_t$ is a proper **weighted average** of past squared gradients. Old information is exponentially forgotten, so the effective learning rate doesn't vanish.

The update is the same as AdaGrad:

$$\Delta \vec{w}_t = \frac{\eta}{\sqrt{\vec{s}_t} + \epsilon} \odot \nabla L_t$$

6. Adam: Combining Momentum and Adaptive Rates

Adam combines the best of momentum (direction smoothing) and RMSProp (per-parameter step sizes).

Two State Vectors

$$\vec{v}_t = (1 - \beta_1) \nabla L_t + \beta_1 \vec{v}_{t-1} \quad (\text{momentum-like: weighted average of gradients})$$

$$\vec{s}_t = (1 - \beta_2) |\nabla L_t|^2 + \beta_2 \vec{s}_{t-1} \quad (\text{RMSProp-like: weighted average of squared gradients})$$

Bias Correction

In early iterations, the state vectors are biased toward zero (since they're initialized to zero). Adam corrects this:

$$\hat{v}_t = \frac{\vec{v}_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{\vec{s}_t}{1 - \beta_2^t}$$

Update

$$\Delta \vec{w}_t = \eta \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \epsilon}$$

Default Hyperparameters

Parameter	Typical Value	Role
η	10^{-3}	Global learning rate
β_1	0.9	Momentum decay
β_2	0.999	Squared gradient decay
ϵ	10^{-8}	Numerical stability

Workout: At iteration $t = 1$ with $\beta_1 = 0.9$, $\beta_2 = 0.999$, the raw first moment is $\vec{v}_1 = 0.1 \nabla L_1$ and the raw second moment is $\vec{s}_1 = 0.001 |\nabla L_1|^2$. Compute the bias-corrected estimates.

Solution:

$$\hat{v}_1 = \frac{0.1 \nabla L_1}{1 - 0.9^1} = \frac{0.1 \nabla L_1}{0.1} = \nabla L_1$$

$$\hat{s}_1 = \frac{0.001 |\nabla L_1|^2}{1 - 0.999^1} = \frac{0.001 |\nabla L_1|^2}{0.001} = |\nabla L_1|^2$$

The bias correction recovers the true gradient and squared gradient magnitudes in the first iteration.

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, betas=(0.9, 0.999))
```

7. Optimizer Comparison Summary

Optimizer	Key Idea	Drawback
Vanilla SGD	Follow negative gradient	Slow, noisy
SGD + Momentum	Exponential average of past gradients	Can overshoot
Nesterov	Look-ahead gradient reduces overshooting	Slightly more expensive
AdaGrad	Per-parameter LR from cumulative gradient ²	LR vanishes over time
RMSProp	AdaGrad + exponential moving average	No momentum
Adam	Momentum + RMSProp + bias correction	Most hyperparameters

Modern default: Adam is the most popular choice. Start with Adam and $\eta = 10^{-3}$ unless you have a specific reason to use something else.

8. Regularization

Overfitting vs. Underfitting

	Training Loss	Test Loss	Diagnosis
Underfitting	High	High	Model too simple
Good fit	Low	Low	Model just right
Overfitting	Very low	High	Model too complex

Overfitting occurs when the network has enough capacity to **memorize** training data (including its noise) rather than learning general patterns.

The Regularized Loss

$$L_{\text{total}}(\theta) = L(\theta) + \lambda R(\theta)$$

where $R(\theta)$ penalizes complexity and λ controls the strength of regularization.

9. L2 Regularization (Weight Decay)

$$R(\theta) = \|\vec{w}\|^2 + \|\vec{b}\|^2$$

The overall objective becomes:

$$L_{\text{total}} = \sum_{i=0}^{n-1} L^{(i)}(\vec{y}^{(i)}, \bar{y}^{(i)}) + \lambda \left(\|\vec{w}\|^2 + \|\vec{b}\|^2 \right)$$

Effect: Pushes all weights toward zero. The gradient of the regularization term is $2\lambda\vec{w}$, which subtracts a fraction of each weight at every update — hence the name **weight decay**.

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01,
weight_decay=1e-4)
```

10. L1 Regularization

$$R(\theta) = |\vec{w}| + |\vec{b}|$$

L1 vs. L2: Sparsity

Property	L1	L2
Penalty	$\ w\ $	w^2
Gradient	± 1 (constant)	$2w$ (proportional to w)
Near zero	Constant push $\rightarrow$ reaches zero	Push shrinks $\rightarrow$ approaches but never reaches zero
Result	Sparse weights (many exact zeros)	Dense weights (small but nonzero)

Nuanced point: L1's constant gradient magnitude means it pushes equally hard regardless of weight magnitude. A weight at $w = 0.001$ gets the same push as $w = 100$. L2's proportional gradient means the push weakens as w approaches zero, so weights shrink but rarely become exactly zero. This makes L1 useful for **feature selection** — irrelevant features get exactly zero weights.

11. Bayesian View: MLE and MAP

MLE (Maximum Likelihood Estimation)

Model each training example's probability as:

$$p(T^{(i)}|\theta) \propto e^{-L^{(i)}(\vec{y}^{(i)}, \vec{y}^{(i)})}$$

Maximizing the joint likelihood $p(T|\theta) = \prod_i p(T^{(i)}|\theta)$ is equivalent to minimizing $L(\theta)$ — the **unregularized** loss.

MAP (Maximum A Posteriori)

Apply Bayes' theorem and assume a prior $p(\theta) \propto e^{-\lambda R(\theta)}$:

$$\theta^* = \arg \max_{\theta} p(\theta|T) = \arg \max_{\theta} p(T|\theta) p(\theta) = \arg \min_{\theta} [L(\theta) + \lambda R(\theta)]$$

This is the **regularized** loss. The Bayesian prior acts as regularization — it encodes our preference for "simple" solutions.

Key insight: L2 regularization corresponds to a Gaussian prior on the weights. L1 regularization corresponds to a Laplace prior.

12. Dropout

Concept

During each training iteration, randomly set a fraction of neuron outputs to zero with probability $(1 - p)$. During inference, all neurons are active but outputs are scaled by p .

Why It Works

1. **Prevents co-adaptation:** Neurons can't rely on specific other neurons always being present
2. **Ensemble effect:** A network with n dropout-able neurons simulates 2^n subnetworks. Training with dropout is like training an ensemble and averaging their predictions.
3. **Spreads representations:** Forces the network to use all neurons rather than concentrating information in a few

Workout: A layer has 3 nodes with dropout probability $p = 0.5$ each. How many distinct subnetworks does this simulate?

Solution: Each node is independently on or off: $2^3 = 8$ subnetworks. With probability $p_k = 0.5$ for each node, all 8 subnetworks are equally likely ($P = 0.5^3 = 0.125$ each).

PyTorch Implementation

```
class ModelWithDropout(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Dropout(p=0.5),
            nn.Linear(hidden_size, output_size),
        )

    def forward(self, x):
        return self.net(x)

model = ModelWithDropout(2, 64, 3)
model.train()  # dropout active during training
model.eval()   # dropout disabled during inference
```

Key Takeaways

1. **Momentum** averages past gradients to amplify the consistent downhill direction and dampen noise
2. **Nesterov** look-ahead reduces overshooting near the minimum
3. **AdaGrad** adapts learning rates per parameter but suffers from vanishing LR
4. **RMSProp** fixes AdaGrad with exponential moving average
5. **Adam** = momentum + RMSProp + bias correction — the modern default optimizer

6. **L2 regularization** (weight decay) penalizes large weights → dense but small weights
7. **L1 regularization** penalizes absolute weights → sparse weights (feature selection)
8. **MLE ↔ unregularized loss, MAP ↔ regularized loss** — a Bayesian perspective
9. **Dropout** randomly disables neurons during training → simulates an ensemble of subnetworks, preventing overfitting